

DE_rendezvous_cobots_Javier_Arambarri

May 22, 2026

1 Evolución Diferencial para el rendezvous de dos cobots cooperativos

Notebook didáctico para resolver una versión simplificada del problema de rendezvous mediante **Evolución Diferencial (DE/rand/1/bin)**.

La solución candidata se codifica como:

[x=(p_{Ax},p_{Ay},p_{Az},s_A,s_B,s_{rail},p_{rail})]

donde $(p_B = p_A + (0.20, 0, 0))$. Las orientaciones se consideran fijas, como en la versión simplificada del trabajo, para centrar el ejemplo en el algoritmo evolutivo.

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from dataclasses import dataclass
from typing import Tuple, Dict

try:
    from scipy.stats import kruskal, mannwhitneyu
    SCIPY_AVAILABLE = True
except Exception:
    SCIPY_AVAILABLE = False
```

1.1 1. Parámetros del problema simplificado

```
[ ]: # Geometría simplificada
R_A = 0.762
R_B = 0.762
HEIGHT = 1.029
D_EF = 0.20
BASE_B_X = 0.75
RAIL_MIN, RAIL_MAX = 0.0, 0.70

# Velocidades nominales
V_ARM_A = 0.60
V_ARM_B = 0.55
V_RAIL = 0.35
```

```

# Tolerancia de sincronización
DELTA_T_MAX = 0.05

# Penalizaciones
LAMBDA_SYNC = 50.0
LAMBDA_FEAS = 1000.0

# Variables: pAx, pAy, pAz, sA, sB, sRail, pRail
LOWER = np.array([-0.20, -0.60, 0.05, 0.05, 0.05, 0.05, RAIL_MIN])
UPPER = np.array([ 0.70,  0.60, 0.95, 1.00, 1.00, 1.00, RAIL_MAX])
DIM = len(LOWER)

P_A_START = np.array([0.0, -0.35, 0.30])
P_B_START_WORLD = np.array([BASE_B_X, 0.35, 0.30])

```

1.2 2. Funciones auxiliares

El modelo geométrico aproxima el espacio de trabajo de cada robot mediante cilindros. En el proyecto real, estas funciones deberían reemplazarse por comprobaciones de cinemática inversa, límites articulares, manipulabilidad y colisiones.

```

[ ]: def pA_from_x(x):
    return x[:3]

def pB_from_x(x):
    return pA_from_x(x) + np.array([D_EF, 0.0, 0.0])

def rail_from_x(x):
    return x[6]

def speeds_from_x(x):
    return x[3], x[4], x[5]

def workspace_violation_A(pA):
    radial = max(0.0, np.sqrt(pA[0]**2 + pA[1]**2) - R_A)
    z_low = max(0.0, -pA[2])
    z_high = max(0.0, pA[2] - HEIGHT)
    return radial**2 + z_low**2 + z_high**2

def workspace_violation_B(pB, pRail):
    base_x = BASE_B_X + pRail
    radial = max(0.0, np.sqrt((pB[0] - base_x)**2 + pB[1]**2) - R_B)
    z_low = max(0.0, -pB[2])
    z_high = max(0.0, pB[2] - HEIGHT)
    rail_low = max(0.0, RAIL_MIN - pRail)
    rail_high = max(0.0, pRail - RAIL_MAX)

```

```

    return radial**2 + z_low**2 + z_high**2 + rail_low**2 + rail_high**2

def feasibility_violation(x):
    pA = pA_from_x(x)
    pB = pB_from_x(x)
    return workspace_violation_A(pA) + workspace_violation_B(pB, rail_from_x(x))

def is_feasible(x, tol=1e-10):
    return feasibility_violation(x) <= tol

def clip_bounds(x):
    return np.minimum(np.maximum(x, LOWER), UPPER)

def repair_solution(x):
    x = clip_bounds(x.copy())
    # Aproximación: situar el raíl para acercar la base de B al punto pB
    pB = pB_from_x(x)
    x[6] = np.clip(pB[0] - BASE_B_X, RAIL_MIN, RAIL_MAX)
    return clip_bounds(x)

```

1.3 3. Modelo de tiempos y función objetivo penalizada

```

[ ]: def times(x):
    x = repair_solution(x)
    pA = pA_from_x(x)
    pB = pB_from_x(x)
    sA, sB, sRail = speeds_from_x(x)
    pRail = rail_from_x(x)

    sA = max(sA, 1e-6)
    sB = max(sB, 1e-6)
    sRail = max(sRail, 1e-6)

    length_A = np.linalg.norm(pA - P_A_START)

    base_B = np.array([BASE_B_X + pRail, 0.0, 0.0])
    start_B_relative = P_B_START_WORLD - np.array([BASE_B_X, 0.0, 0.0])
    length_B = np.linalg.norm((pB - base_B) - start_B_relative)

    tA = length_A / (sA * V_ARM_A)
    tB = length_B / (sB * V_ARM_B) + abs(pRail) / (sRail * V_RAIL)
    return tA, tB

def sync_penalty(delta_t):
    if delta_t <= DELTA_T_MAX:
        return (delta_t / DELTA_T_MAX)**2
    return 1.0 + ((delta_t - DELTA_T_MAX) / DELTA_T_MAX)**2

```

```

def objective_components(x):
    x = repair_solution(x)
    tA, tB = times(x)
    delta = abs(tA - tB)
    tmax = max(tA, tB)
    psync = sync_penalty(delta)
    pfeas = feasibility_violation(x)
    J = tmax + LAMBDA_SYNC * psync + LAMBDA_FEAS * pfeas
    return {
        "J": J,
        "Tmax": tmax,
        "tA": tA,
        "tB": tB,
        "delta_t": delta,
        "Psync": psync,
        "Pfeas": pfeas,
        "feasible": is_feasible(x),
    }

def objective(x):
    return objective_components(x)["J"]

```

1.4 4. Inicialización de población

```

[ ]: def sample_candidate(rng):
    for _ in range(5000):
        x = np.array([
            rng.uniform(0.0, 0.60),      # pAx
            rng.uniform(-0.50, 0.50),    # pAy
            rng.uniform(0.10, 0.90),     # pAz
            rng.uniform(0.20, 1.00),     # sA
            rng.uniform(0.20, 1.00),     # sB
            rng.uniform(0.20, 1.00),     # sRail
            rng.uniform(RAIL_MIN, RAIL_MAX)
        ])
        x = repair_solution(x)
        if is_feasible(x):
            return x
    return repair_solution(rng.uniform(LOWER, UPPER))

def initialize_population(rng, pop_size):
    return np.array([sample_candidate(rng) for _ in range(pop_size)])

```

1.5 5. Algoritmo DE/rand/1/bin

```
[ ]: @dataclass
class DEParams:
    pop_size: int = 60
    generations: int = 150
    F: float = 0.75
    CR: float = 0.90

def differential_evolution_rendevvous(params=DEParams(), seed=0,
    ↪store_history=True):
    rng = np.random.default_rng(seed)
    pop = initialize_population(rng, params.pop_size)
    fitness = np.array([objective(ind) for ind in pop])

    history = {
        "best_J": [],
        "mean_J": [],
        "best_Tmax": [],
        "best_delta_t": [],
        "feasible_ratio": [],
        "populations": [],
    }

    for gen in range(params.generations):
        new_pop = pop.copy()
        new_fit = fitness.copy()

        for i in range(params.pop_size):
            candidates = [j for j in range(params.pop_size) if j != i]
            r1, r2, r3 = rng.choice(candidates, size=3, replace=False)

            mutant = pop[r1] + params.F * (pop[r2] - pop[r3])
            mutant = repair_solution(mutant)

            mask = rng.random(DIM) < params.CR
            mask[rng.integers(DIM)] = True
            trial = np.where(mask, mutant, pop[i])
            trial = repair_solution(trial)

            trial_fit = objective(trial)
            if trial_fit <= fitness[i]:
                new_pop[i] = trial
                new_fit[i] = trial_fit

        pop = new_pop
        fitness = new_fit
```

```

        best_idx = np.argmin(fitness)
        comp = objective_components(pop[best_idx])
        history["best_J"].append(comp["J"])
        history["mean_J"].append(np.mean(fitness))
        history["best_Tmax"].append(comp["Tmax"])
        history["best_delta_t"].append(comp["delta_t"])
        history["feasible_ratio"].append(np.mean([is_feasible(ind) for ind in pop]))

    if store_history:
        history["populations"].append(pop.copy())

    best_idx = np.argmin(fitness)
    best_x = pop[best_idx]
    return best_x, objective_components(best_x), history

```

1.6 6. Ejecución básica

```

[ ]: params = DEParams(pop_size=60, generations=150, F=0.75, CR=0.90)
best_x, best_comp, hist = differential_evolution_rendezvous(params, seed=1)

pd.DataFrame([
    "pA": np.round(pA_from_x(best_x), 4),
    "pB": np.round(pB_from_x(best_x), 4),
    "sA": round(best_x[3], 4),
    "sB": round(best_x[4], 4),
    "sRail": round(best_x[5], 4),
    "pRail": round(best_x[6], 4),
    **{k: round(v, 6) if isinstance(v, float) else v for k, v in best_comp.items()}
])

```

1.7 7. Curvas de convergencia

```

[ ]: plt.figure(figsize=(8,5))
plt.plot(hist["best_J"], label="Mejor J")
plt.plot(hist["mean_J"], label="J medio")
plt.yscale("log")
plt.xlabel("Generación")
plt.ylabel("Función penalizada")
plt.title("Convergencia de DE")
plt.grid(True)
plt.legend()
plt.show()

plt.figure(figsize=(8,5))

```

```

plt.plot(hist["best_Tmax"], label="Mejor Tmax")
plt.xlabel("Generación")
plt.ylabel("Tmax (s)")
plt.title("Evolución del tiempo máximo")
plt.grid(True)
plt.legend()
plt.show()

plt.figure(figsize=(8,5))
plt.plot(hist["best_delta_t"], label="|tA - tB|")
plt.axhline(DELTA_T_MAX, linestyle="--", label="Tolerancia")
plt.xlabel("Generación")
plt.ylabel("Desincronización (s)")
plt.title("Evolución de la sincronización")
plt.grid(True)
plt.legend()
plt.show()

```

1.8 8. Visualización del espacio de búsqueda

```

[ ]: def plot_workspace_xy(pop=None, best_x=None, title="Espacio XY"):
    fig, ax = plt.subplots(figsize=(7,6))

    ax.add_patch(plt.Circle((0,0), R_A, fill=False, linewidth=2, label="Alcance_
    ↪A"))

    rail = rail_from_x(best_x) if best_x is not None else 0.35
    ax.add_patch(plt.Circle((BASE_B_X + rail,0), R_B, fill=False, linewidth=2,
    ↪label="Alcance B"))

    ax.plot([BASE_B_X + RAIL_MIN, BASE_B_X + RAIL_MAX], [-0.72, -0.72],
    ↪linewidth=4, label="Raíl")

    if pop is not None:
        pAs = np.array([pA_from_x(ind) for ind in pop])
        pBs = np.array([pB_from_x(ind) for ind in pop])
        ax.scatter(pAs[:,0], pAs[:,1], s=20, label="$p_A$")
        ax.scatter(pBs[:,0], pBs[:,1], s=20, marker="x", label="$p_B$")

    if best_x is not None:
        pA, pB = pA_from_x(best_x), pB_from_x(best_x)
        ax.scatter([pA[0]], [pA[1]], s=150, marker="*", label="Mejor $p_A$")
        ax.scatter([pB[0]], [pB[1]], s=150, marker="*", label="Mejor $p_B$")
        ax.plot([pA[0], pB[0]], [pA[1], pB[1]], linestyle="--")

    ax.set_aspect("equal", adjustable="box")
    ax.set_xlim(-0.9, 1.8)

```

```

ax.set_ylim(-0.9, 0.9)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title(title)
ax.grid(True)
ax.legend()
plt.show()

plot_workspace_xy(hist["populations"][0], title="Población inicial")
plot_workspace_xy(hist["populations"][-1], best_x=best_x, title="Población_
↪final")

```

```

[ ]: def plot_workspace_3d(pop, best_x=None):
    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(111, projection="3d")

    pAs = np.array([pA_from_x(ind) for ind in pop])
    pBs = np.array([pB_from_x(ind) for ind in pop])

    ax.scatter(pAs[:,0], pAs[:,1], pAs[:,2], s=25, label="$p_A$")
    ax.scatter(pBs[:,0], pBs[:,1], pBs[:,2], s=25, marker="x", label="$p_B$")

    if best_x is not None:
        pA, pB = pA_from_x(best_x), pB_from_x(best_x)
        ax.scatter([pA[0]], [pA[1]], [pA[2]], s=160, marker="*", label="Mejor_
↪$p_A$")
        ax.scatter([pB[0]], [pB[1]], [pB[2]], s=160, marker="*", label="Mejor_
↪$p_B$")
        ax.plot([pA[0], pB[0]], [pA[1], pB[1]], [pA[2], pB[2]], linestyle="--")

    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("z")
    ax.set_title("Puntos de rendezvous en 3D")
    ax.legend()
    plt.show()

plot_workspace_3d(hist["populations"][-1], best_x=best_x)

```

1.9 9. Animación opcional

```

[ ]: from matplotlib.animation import FuncAnimation
    from IPython.display import HTML

    def animate_de_xy(hist, every=4):
        populations = hist["populations"][::every]
        fig, ax = plt.subplots(figsize=(7,6))

```

```

ax.add_patch(plt.Circle((0,0), R_A, fill=False, linewidth=2))
ax.add_patch(plt.Circle((BASE_B_X + 0.35,0), R_B, fill=False, linewidth=2))
ax.plot([BASE_B_X + RAIL_MIN, BASE_B_X + RAIL_MAX], [-0.72, -0.72],
↪linewidth=4)

scat_A = ax.scatter([], [], s=25, label="$p_A$")
scat_B = ax.scatter([], [], s=25, marker="x", label="$p_B$")
best_scat = ax.scatter([], [], s=150, marker="*", label="Mejor $p_A$")

ax.set_aspect("equal", adjustable="box")
ax.set_xlim(-0.9, 1.8)
ax.set_ylim(-0.9, 0.9)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.grid(True)
ax.legend()

def init():
    scat_A.set_offsets(np.empty((0, 2)))
    scat_B.set_offsets(np.empty((0, 2)))
    best_scat.set_offsets(np.empty((0, 2)))
    return scat_A, scat_B, best_scat

def update(frame):
    pop = populations[frame]
    pAs = np.array([pA_from_x(ind) for ind in pop])
    pBs = np.array([pB_from_x(ind) for ind in pop])
    fit = np.array([objective(ind) for ind in pop])
    best = pAs[np.argmin(fit)]

    scat_A.set_offsets(pAs[:, :2])
    scat_B.set_offsets(pBs[:, :2])
    best_scat.set_offsets(best[:2].reshape(1, 2))
    ax.set_title(f"DE - generación {frame * every}")
    return scat_A, scat_B, best_scat

anim = FuncAnimation(fig, update, frames=len(populations), init_func=init,
↪interval=250, blit=True)
plt.close(fig)
return HTML(anim.to_jshtml())

display(animate_de_xy(hist))

```

1.10 10. Ajuste de parámetros

```
[ ]: def run_grid(n_runs=15):
    configs = [
        {"F": 0.4, "CR": 0.5},
        {"F": 0.6, "CR": 0.7},
        {"F": 0.75, "CR": 0.9},
        {"F": 0.9, "CR": 0.9},
    ]
    rows = []
    for cfg in configs:
        for seed in range(n_runs):
            params = DEParams(pop_size=60, generations=120, F=cfg["F"],
                               CR=cfg["CR"])
            bx, comp, _ = differential_evolution_rendezvous(params, seed=seed,
                                                           store_history=False)
            rows.append({
                "F": cfg["F"],
                "CR": cfg["CR"],
                "seed": seed,
                "J": comp["J"],
                "Tmax": comp["Tmax"],
                "delta_t": comp["delta_t"],
                "feasible": comp["feasible"],
                "sA": bx[3],
                "sB": bx[4],
                "sRail": bx[5],
                "pRail": bx[6],
            })
    return pd.DataFrame(rows)

grid_results = run_grid(n_runs=15)
grid_results.groupby(["F", "CR"])[["J", "Tmax", "delta_t"]].agg(["mean", "std",
                                                                "min", "median"])
```

```
[ ]: labels, data = [], []
for (F, CR), group in grid_results.groupby(["F", "CR"]):
    labels.append(f"F={F}, CR={CR}")
    data.append(group["Tmax"].values)

plt.figure(figsize=(8,5))
plt.boxplot(data, labels=labels)
plt.xticks(rotation=30)
plt.ylabel("Tmax final")
plt.title("Comparativa de parámetros de DE")
plt.grid(True)
plt.show()
```

```

if SCIPY_AVAILABLE:
    groups = [g["Tmax"].values for _, g in grid_results.groupby(["F", "CR"])]
    stat, pvalue = kruskal(*groups)
    print(f"Kruskal-Wallis: statistic={stat:.4f}, p-value={pvalue:.6f}")
else:
    print("SciPy no está disponible; se omite el contraste.")

```

1.11 11. Comparación con búsqueda aleatoria

```

[ ]: def random_search(n_evals=60*120, seed=0):
    rng = np.random.default_rng(seed)
    best_x = None
    best_J = np.inf
    for _ in range(n_evals):
        x = sample_candidate(rng)
        J = objective(x)
        if J < best_J:
            best_J = J
            best_x = x.copy()
    return best_x, objective_components(best_x)

def compare_de_random(n_runs=20):
    rows = []
    for seed in range(n_runs):
        bx, comp, _ = differential_evolution_rendezvous(
            DEParams(pop_size=60, generations=120, F=0.75, CR=0.9),
            seed=seed,
            store_history=False
        )
        rows.append({"method": "DE", "seed": seed, "Tmax": comp["Tmax"], "J":
↪comp["J"], "delta_t": comp["delta_t"]})

        bx, comp = random_search(n_evals=60*120, seed=seed)
        rows.append({"method": "Random Search", "seed": seed, "Tmax":
↪comp["Tmax"], "J": comp["J"], "delta_t": comp["delta_t"]})

    return pd.DataFrame(rows)

baseline_results = compare_de_random(n_runs=20)
baseline_results.groupby("method")[["Tmax", "J", "delta_t"]].agg(["mean",
↪"std", "min", "median"])

```

```

[ ]: plt.figure(figsize=(7,5))
baseline_results.boxplot(column="Tmax", by="method")
plt.suptitle("")
plt.title("DE frente a búsqueda aleatoria")

```

```

plt.ylabel("Tmax final")
plt.grid(True)
plt.show()

if SCIPY_AVAILABLE:
    de_values = baseline_results[baseline_results["method"] == "DE"]["Tmax"].
↪values
    rs_values = baseline_results[baseline_results["method"] == "Random_
↪Search"]["Tmax"].values
    stat, pvalue = mannwhitneyu(de_values, rs_values, alternative="two-sided")
    print(f"Mann-Whitney U: statistic={stat:.4f}, p-value={pvalue:.6f}")

```

1.12 12. Adaptación al proyecto real

Para aproximar este notebook al problema real del trabajo:

1. Sustituir el modelo cilíndrico por cinemática inversa real.
2. Añadir comprobación de singularidades mediante manipulabilidad o valor singular mínimo del Jacobiano.
3. Sustituir `times(x)` por un estimador real de tiempo de trayectoria.
4. Reintroducir orientaciones variables si se desea resolver el problema completo.
5. Normalizar cuaterniones después de cada mutación diferencial.
6. Comparar DE con el Temple Simulado usado anteriormente mediante 30 ejecuciones independientes.